Computer Networks 274 (2026) 111806

Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet

MGDA: A provenance graph-based framework for threat detection and attack scenario reconstruction

Mengjiao Cui [a,b], Zhengwei Jiang [a,b], Shuai Li [a,b], Chunyan Ma [a,b], Kai Zhang [a], Peian Yang [a,*], Huamin Feng [b,c]

[a] Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China
[b] School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China
[c] Beijing Electronic Science and Technology Institute, Beijing, China

ARTICLE INFO

Keywords:
Attack detection
Attack investigation
Provenance graph
Graph representation learning

ABSTRACT

Advanced persistent threat (APT) attacks are sophisticated, stealthy, and persistent, posing significant challenges to timely detection and investigation in modern network environments. Provenance graph analysis has become an important method for APT detection due to its ability to capture detailed causal relationships among system entities. However, existing methods suffer from several limitations: (1) lack of labeled attack data, (2) lack of high-level semantics in attack scenario reconstruction, and (3) high computational overhead limiting practical deployment. In this paper, we propose MGDA, a self-supervised method for effective and accurate threat detection as well as interpretable attack scenario reconstruction. MGDA introduces a multi-view masked graph autoencoder that jointly captures deep semantic features and structural patterns, enabling accurate detection of stealthy and unknown attacks. In the reconstruction phase, MGDA combines contextual analysis with rule-based attack pattern matching to produce attack scenario graphs that incorporate high-level semantics. We evaluate MGDA on three widely used datasets, including both real-world and simulated network attacks. The results demonstrate that MGDA achieves an average precision of 97.58 % and F1-score of 98.03 % in threat detection, outperforming state-of-the-art approaches. In addition, the automatically reconstructed scenario graphs help identify potential multi-step attacks and their stages, aiding analysts in conducting efficient network attack investigations.

1. Introduction

Advanced Persistent Threats (APTs) are sophisticated and persistent attacks that leverage a variety of techniques to conduct highly stealthy, multi-stage, long-duration network attacks on the target. As one of the most severe threats to modern network security, APTs typically target critical hosts in large enterprises and government infrastructures. Attackers often emulate trusted benign processes to hide their attacks and use novel techniques to evade detection and prolong their presence on the target system. They are difficult to detect using traditional detection methods. Statistically, malicious behaviors are deeply buried within massive volumes of benign system events, with attack data sometimes accounting for less than 0.001 % of the total audit data volume. Such an extremely low percentage makes it difficult for traditional approaches based on anomaly detection to function effectively. In addition, insufficient modeling of system behaviors may cause APT activities to closely resemble normal behavior in the feature space, thereby reducing the discriminative power of detection methods. Existing intrusion detection systems perform poorly against such highly customized and dynamically evolving APT attacks [1]. In recent years, data provenance analysis on provenance graphs has become a widely used approach for APT detection in network security. Data provenance can be represented as a directed graph, called a provenance graph, where nodes represent subjects (e.g., processes) and objects (e.g., files and network sockets) and edges denote the interactions between these entities. The provenance graph preserves the causal relationship between system activities, which can effectively analyze the cause of threat events, attack paths, and their impacts, providing high detection efficiency and robustness for threat detection and investigation [2].

APT detection based on provenance graphs can be categorized into the following three types. The first is rule-based detection [3–7], which discovers attacks through rule matching. While effective in capturing known attack patterns, these methods incur high manual costs in rule engineering and maintenance, lack adaptability to novel threats, and often result in a high false positive rate, contributing to alert fatigue and decreased operational efficiency. Statistical-based approaches [8,9] iden-

* Corresponding author.
E-mail addresses: cuimengjiao@iie.ac.cn (M. Cui), jiangzhengwei@iie.ac.cn (Z. Jiang), lishuai2023@iie.ac.cn (S. Li), machunyan@iie.ac.cn (C. Ma), zhangkai0216@iie.ac.cn (K. Zhang), yangpeian@iie.ac.cn (P. Yang), fenghm@besti.edu.cn (H. Feng).

https://doi.org/10.1016/j.comnet.2025.111806
Received 8 July 2025; Received in revised form 1 October 2025; Accepted 22 October 2025
Available online 29 October 2025
1389-1286/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

tify system entities, interactions, and communities by analyzing their rarity and anomaly scores. However, this approach is limited to shallow feature (such as node in-degree and out-degree) extraction from provenance graphs, making it difficult to effectively extract the rich contextual information hidden between nodes. As a result, the fine-grained semantic associations between system entities and behaviors cannot be well modeled. This often leads to high false positive rates when dealing with complex or stealthy attacks. Learning-based approaches [10,11], such as machine learning (ML) and deep learning (DL) models, are currently popular due to their ability to automatically learn behavior patterns from data.

After detection, existing approaches often attempt to reconstruct attack scenarios by correlating anomalous activities based on simple temporal relations. For example, Kairos [12] applies community detection and aggregation techniques to group provenance data into meaningful subgraphs, while Flash [13] constructs attack evolution graphs through causal tracing to link different attack stages. These methods help analysts organize massive event data into more manageable structures. However, they only address the symptoms rather than the root cause: the reconstructed graphs are often oversized and redundant, and more critically, they lack the abstraction of higher-level semantics such as attack techniques and stages. Without such semantics, it is difficult to connect fine-grained system events to broader multi-stage campaigns, which significantly limits the interpretability and effectiveness of attack scenario reconstruction.

While promising, these methods still face significant limitations when applied to real-world attack detection and investigation. The practical application of such approaches remains challenging due to the following key issues: (1) Lack of labeled attack data. Most learning-based methods rely on supervised training and require sufficient labeled attack samples. In real-world settings, however, attack data is scarce, and manual labeling is expensive and difficult to scale, which limits the model's ability to detect unknown or novel attacks. (2) Lack of high-level semantics in attack scenario reconstruction. Existing methods often reconstruct attack scenarios by linking low-level system events based on simple correlations (e.g., timestamps). This approach produces overly large and redundant graphs and fails to capture higher-level semantics, such as attack stages and techniques, which are essential for understanding multi-stage attacks. As a result, the reconstructed scenarios are often incomplete and less interpretable, increasing both the cost and difficulty of attack investigation. Moreover, existing works frequently struggle to balance the accuracy of attack detection with the completeness and interpretability of the reconstructed scenarios. (3) High computational overhead. The scale and complexity of system audit logs are rapidly increasing, resulting in massive and often redundant provenance graphs. Some DL methods are impractical for real-world deployment due to high time and memory overhead, making it difficult to efficiently analyze the provenance graph and perform real-time threat detection.

To address these challenges, we propose MGDA, a self-supervised method for APT detection and attack scenario reconstruction. MGDA leverages multi-view masked graph representation learning and outlier detection to identify malicious entities from provenance graphs. It then applies contextual analysis and rule-based matching to reconstruct attack scenarios. MGDA first constructs the provenance graph from audit logs. It then employs a graph representation module to perform both feature and structure reconstruction in a self-supervised manner. A lightweight anomaly detection module is applied to identify suspicious entities. Finally, based on the detected anomalous nodes, MGDA performs contextual analysis and rule-based matching of tactics, techniques, and procedures (TTPs) to reconstruct attack scenarios with rich semantics such as attack stages. Our approach not only accurately detects malicious entities, but also reconstructs interpretable attack scenarios that facilitate analysts' understanding and investigation.

We performed a comprehensive evaluation of MGDA using datasets widely adopted by the research community, including StreamSpot [14], Unicorn Wget [10], and the DARPA Transparent Computing E3 datasets [15]. These datasets encompass a broad spectrum of attack scenarios and system behaviors. Evaluation results show that MGDA achieves an average precision of 97.58 % and an F1-score of 98.03 %. Furthermore, MGDA maintains low computational overhead and demonstrates up to 3× faster training speed compared to existing methods, highlighting its practicality in real-world deployment scenarios.

The primary contributions of our work are as follows:

- We propose MGDA, a self-supervised framework that tightly integrates anomaly detection and attack investigation. It models benign behaviors to reduce reliance on labeled attack data and effectively detect stealthy or previously unseen attacks.
- We employ a novel multi-view masked graph auto-encoder to capture deep hidden semantics among system entities, combined with a sample-based structure reconstruction strategy to learn structural patterns. These designs jointly produce high-quality embeddings, improving detection accuracy.
- We propose a TTP rule-based attack scenario reconstruction method inspired by the ATT&CK framework. It can generate attack scenario graphs with high-level contextual semantics, effectively supporting the tracking and analysis of complex attacks.
- We conduct a comprehensive evaluation of MGDA on three widely used datasets. Results demonstrate that MGDA outperforms most existing methods in APT detection while maintaining low computational overhead.

2. Related work

MGDA is mainly related to two major research areas: APT detection and attack investigation.

2.1. APT detection

Rule-based approaches rely on analyzing historical attacks to construct predefined rules, which are then used to match malicious behaviors for APT detection. For instance, Holmes [4] designs a high-level scenario graph (HSG) by leveraging threat subgraphs constructed by security experts as knowledge graphs. It defines 16 types of TTP rules across seven APT attack phases, mapping low-level system activities to the MITRE ATT&CK matrix [16]. These rules are matched to system behaviors in provenance graphs via graph matching algorithms, enabling the identification of attack phases. The method relies entirely on expert-defined rules, and the detection results are heavily dependent on security experts. RapSheet [3] integrates detection rules from a commercial Endpoint Detection and Response (EDR) to map attacks to ATT&CK-related tactics, which covers more specific information about the attacking techniques and tactics compared to Holmes. Sleuth [5] and Morse [7] utilize label-based rule definitions to trigger alerts. However, they suffer from label over-propagation, where long-term attacks cause exponential growth in node dependencies, leading to dependency explosion. Poirot [6] manually constructs query graphs from APT attack reports, and uses fuzzy graph alignment techniques for detection. However, its computational complexity grows rapidly with the graph size, limiting its real-time applicability. Overall, the rule engineering and maintenance process for these heuristic methods is costly and labor-intensive, requiring constant updating of the knowledge base and rules. Moreover, since APT attacks often leverage zero-day exploits, rule-based approaches struggle to detect novel or unknown threats, keeping defenders perpetually behind advanced adversaries.

Statistical-based methods [8,17,18] quantify the rarity of statistical entities or behaviors within the provenance graph, assigning suspicion scores to these entities, which ultimately leads to anomaly detection. Priotracker [18] and Nodoze [8] adopt statistical feature-based attack detection methods by prioritizing anomalous events and causal dependencies. PrioTracker assigns higher priority to edges of anomalous events. NoDoze improves on the above by prioritizing entire paths by rarity.

Alerts are initiated for rare paths. These approaches assume that the processes involved in an attack exhibit anomalous behavior. However, they often fail when malicious activities are carefully blended into benign background operations. ProvDetector's [9] rarity-based path selection algorithm identifies potentially covert malicious behavior. These statistical approaches typically extract only shallow features from the provenance graph and often fail to capture the rich semantic relationships among entities, resulting in elevated false positive rates.

Learning-based methods detect APT attacks by training models to recognize activities that are different from normal behavior. These methods often leverage anomaly detection techniques to uncover novel or previously unseen attacks by evaluating deviations in runtime behaviors from established benign execution profiles. Graph-based anomaly detection approaches [10,14] utilize graph kernel techniques to dynamically model system behavior and identify anomalous graphs via clustering-based analysis. Threatrace [19] detects host-based threats in real-time at the granularity of system entities without prior knowledge of attack patterns. It customizes GraphSAGE, an inductive graph neural network, to learn role embeddings of benign entities and detect anomalies by identifying deviations from predicted roles. MAGIC [20] employs a masked graph autoencoder to model the behavior of benign system entities. The model is trained by reconstructing node feature masks and structural features in the graph, and subsequently applies a KNN-based outlier detector to identify anomalous system behaviors. However, the performance of this method is affected by the quality of the training data, and many false alarms may be generated if low quality data is used for training. KAIROS [12] adopts an encoder-decoder graph neural network to capture the temporal evolution of structural changes in provenance graphs, enabling fine-grained assessment of system event anomalies. Leveraging this fine-grained information, KAIROS reconstructs attack traces and generates compact summary graphs that accurately characterize malicious activities in the system audit log stream. Flash [13] utilizes Word2Vec-based semantic encoding to capture entities and temporal sequences, encodes local and global graph structures into node embeddings. A lightweight classifier with GNN and Word2Vec embeddings learns benign node behavior and detects anomalies in real-time.

2.2. Attack investigation

Attack investigation techniques can provide important support to security analysts, not only by helping to verify the validity of alerts, but also by assisting in the reconstruction and interpretation of threat events. Some detection methods can indirectly realize the reconstruction of attack scenarios during deployment. Recent studies [21] attempt to bridge the gap between detection and investigation by reconstructing attack scenarios, improving both the efficiency and interpretability of investigation. APTSHIELD [22] integrates ATT&CK to set entity tags and propagation rules, enabling alert aggregation and attack path recovery via label propagation. Holmes [4] utilizes TTP-based specifications to visualize high-level attack scenarios and relate suspicious flows in audit logs. Other approaches such as RapSheet [3] leverage Tactical Provenance Graphs (TPGs) to offer concise, multi-stage attack visualizations, supporting faster investigation and alert generation. Flash [13] constructs Attack Evolution Graphs (AEGs) by tracing causal relationships, helping analysts associate the various stages of an APT attack efficiently. ATLAS [11] relies on the performance of model learning and has low interpretability in reconstructing attack graphs. Krystal [23] constructs attack graph from provenance graphs by correlating alerts through forward and backward search. In addition, they construct attack query graphs based on external knowledge such as Cyber Threat Intelligence (CTI) reports, public malware documents, etc. It can further use graph queries to identify high-level attack stages in the generated attack graphs, providing richer information to analysts.

The core goal of attack investigation is to mitigate alert fatigue and enable analysts to rapidly obtain a holistic understanding of the threat. Therefore, accuracy, timeliness, and interpretability of investigation techniques are critical factors of their effectiveness. However, many existing approaches struggle to strike a good balance among these aspects. In contrast, our approach generates attack scenario graphs that are both comprehensive and interpretable, while maintaining low false-positive rates during detection.

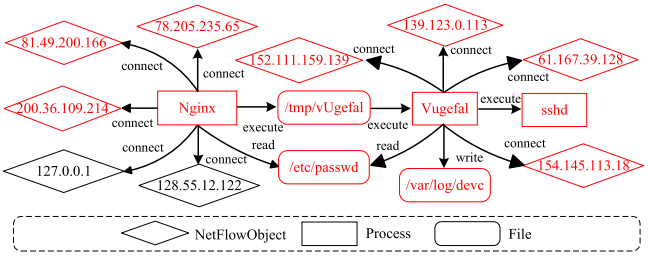


Fig. 1. Attack scenario from DARPA E3 CADETS.

3. Background

3.1. Motivating example

This is an attack case from the DARPA E3 CADETS dataset. The attacker exploits a vulnerable Nginx server on the CADETS FreeBSD host to gain initial access. A shell connection is then established from IP address 81.49.200.166. Through this shell, the attacker downloads a malicious payload to /tmp/vUgefal and executes it with root privileges. The vUgefal process then moves laterally to 61.167.39.128, completes the intrusion, and attempts to inject into the sshd process while downloading a second payload to /var/log/devc. Fig. 1 illustrates our motivating example. Three types of nodes represent system entities—processes, files, and network sockets—while directed edges indicate their interactions.

3.2. Threat model

To build a comprehensive threat model, we make the following assumptions regarding attacker behavior and system characteristics. We focus on APTs that infiltrate target hosts via network channels, preinstalled backdoors, or software vulnerabilities. These attacks are typically highly stealthy and can persist over extended periods. During intrusion, attackers often conceal malicious behavior within legitimate system activities, making it difficult for defenders to distinguish between benign and malicious actions. In addition, attackers may exploit zero-day vulnerabilities, where no prior knowledge or labeled data is available, posing a significant challenge for detection models. They may also adopt living-off-the-land techniques by injecting malicious code into legitimate processes that subsequently spawn new processes to continue malicious operations.

Despite the stealthy nature of these threats, system audit logs provide a complete and traceable record of activities and evidence. Following most prior work in this field [10,13,19], we assume that the underlying operating system, audit framework, and hardware are trustworthy [24]. We further assume that attackers cannot tamper with audit logs, preserving the semantic and structural integrity of the provenance graphs constructed by MGDA.

4. Methodology

In this section, we provide a detailed overview of MGDA's overall design framework and the design of each module. The architecture of MGDA is depicted in Fig. 2. The framework consists of the following components: ① Provenance Graph Construction, ②Graph Representation, ③Threat Detection and ④Attack Reconstruction

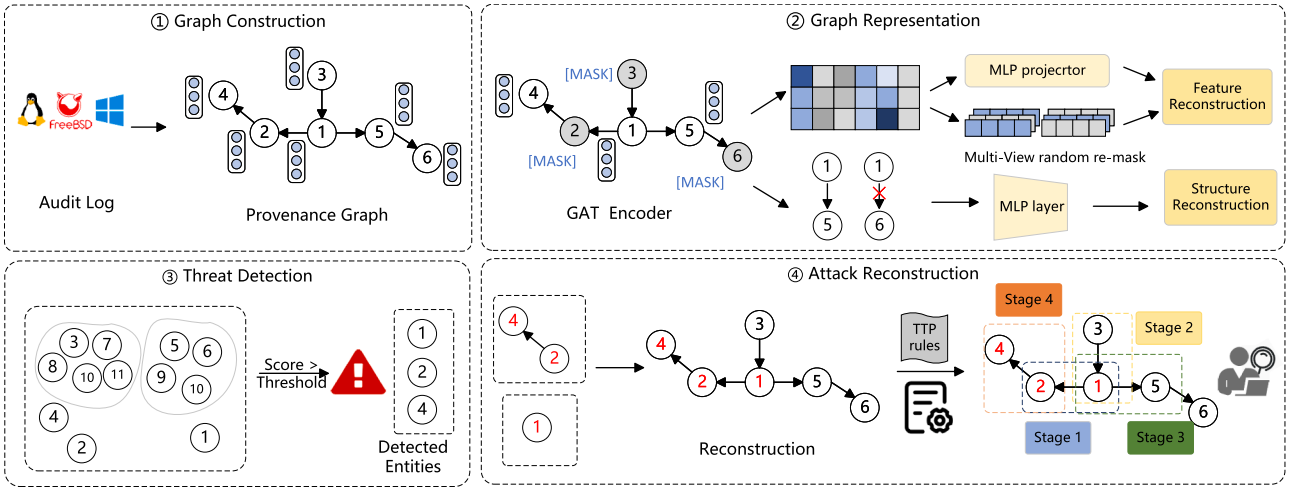


Fig. 2. The framework of MGDA.

Table 1
System entities and dependency relationships.

Subject	Object	Edge Types
Process	Process	Fork, Close…
	File	Read, Write, Open, Execute…
	Socket	Connect, Send, Receive …

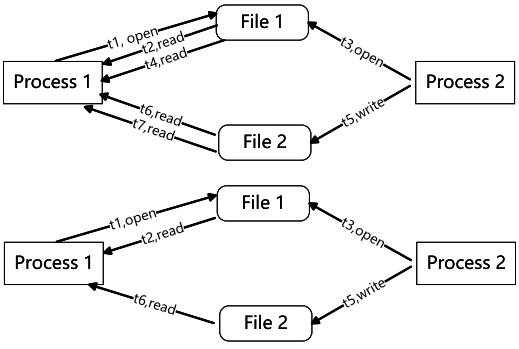


Fig. 3. An example of data compression.

4.1. Provenance graph construction

MGDA collects kernel-level audit logs from various sources (e.g., Windows ETW, Linux Audit, and CamFlow [24]) and transforms them uniformly into a graph structure known as a Provenance Graph (PG). The logs are parsed into structured records. Each log entry (subject, relation, object, timestamp) describes an interaction between system entities, where the subject and object denote the participating entities and the relation specifies the type of interaction. Based on these interactions, MGDA constructs a directed provenance graph, in which nodes represent system entities and edges represent their interactions. The direction of each edge reflects the flow of information. For example, when process A reads file B, an edge is created from node A to node B (A → R-read → B). Table 1 summarizes the entity and relation types considered in MGDA. Fig. 1 shows the provenance graph of a real-world APT attack.

MGDA considers three types of information: node types (e.g., process, file), edge types (e.g., read, execute), and semantic attributes of nodes (e.g., process paths, file names, command-line arguments, source IP addresses). During model training, only node and edge types are used for structural feature embedding to control training complexity. Meanwhile, semantic attributes are stored in a mapping dictionary indexed by node IDs. This design reduces the graph structure training complexity while retaining rich semantic information to support interpretable and context-aware attack scenario reconstruction.

Data compression. Data compression can reduce resource consumption and alleviate cache pressure. However, it is crucial to preserve the data dependency during compression to ensure the integrity and traceability of the results. To address this, we adopt a semantics-preserving compression strategy [25,26] that eliminates redundant events in audit logs. Events are merged only when they are semantically equivalent, meaning the state of the source entity remains unchanged and the destination entity receives multiple identical information flows. For example, as illustrated in Fig. 3, events t2 and t4 both correspond to process1 read file1 at different timestamps. Only the earliest system event (t2) is retained. This compression reduces graph size and redundancy without losing essential behavior semantics. It improves training efficiency and eliminates repeated actions. Moreover, it enables the model to focus on diverse and representative benign behaviors, rather than repeated instances of the same action, thereby enhancing threat detection performance.

Initial embedding. We initialize the embeddings of nodes and edges during the graph construction stage. Unlike prior approaches such as Flash [13] that incorporate semantic attribute encoding using Word2Vec, our approach adopts a simplified yet effective encoding strategy based solely on the one-hot representation of node and edge types. Specifically, each node is encoded by its type (e.g., process, file, socket), and each edge is encoded based on its interaction type (e.g., read, write, execute), thereby enabling lightweight modeling of system interactions. For instance, given five types of edge relations, each edge is represented by a 5-dimensional one-hot vector, where [0, 0, 0, 0, 1] denotes the fifth type and [1, 0, 0, 0, 0] denotes the first.

We empirically find that incorporating semantic attributes of nodes (such as process paths, file names, or command-line parameters) through hash-based [12] embeddings significantly increases the complexity of subsequent representation learning and introduces noise, which in turn harms the model's anomaly detection accuracy. Therefore, we discard semantic attributes during the feature embedding phase and instead preserve them externally for downstream tasks such as attack scenario reconstruction and analyst interpretation.

4.2. Graph representation

To address the scarcity of labeled attack data in real-world environments, we employ a graph self-supervised learning (SSL) technique to derive high-quality embeddings from the original feature graph. Existing masked self-encoders such as GraphMAE [27] have shown

promising results in reconstructing node features. However, their performance heavily depends on the distinguishability of input features and is vulnerable to feature interference. In APT scenarios, malicious nodes often mimic benign behavior through a large number of normal-like operations, obscuring their malicious intent. This behavior reduces the feature distinction between benign and malicious nodes, ultimately degrading the robustness and detection accuracy of the model.

To enhance embedding quality and model robustness, We propose an improved graph representation learning model inspired by Magic [20] and GraphMAE2 [28]. The node feature reconstruction is regularized using multi-view random mask decoding as well as introducing a latent representation prediction strategy that performs reconstruction in the embedding space. In addition, graph structure reconstruction is achieved by the edge connection prediction task, which strengthens the model's ability to capture global topological relationships. The combination of multi-view masking and latent representation learning improves model resilience to noise and enables the capture of implicit semantics and structural patterns among system entities. As a result, the model generates higher quality and more discriminative node embeddings, laying a solid foundation for downstream tasks such as node classification and anomaly detection. The graph representation module consists of three components: node masking, graph encoder, and graph decoder. The encoder and decoder form a masked graph autoencoder.

4.2.1. Node masking

During training, the graph representation module applies masking operations on the nodes in the graph. Edges are not masked because they provide critical relational dependencies among system entities. The masked nodes are obtained by random sampling, similar to the pre-training process of MAE or BERT [29]. Randomly sample some nodes from the graph and replace their corresponding features with mask token [MASK] or all-zero vectors. A relatively high mask ratio (e.g., 50 %) is employed for sampling, which helps to eliminate redundancy and learn robust structural and contextual representations.

Given a graph $G = (V, A, X)$, where V is the node set, A is the edge set and X is the input node feature matrix, we denote GNN encoder as f_E and the decoder as F_D.

$$H = f_E(A, X), \tilde{G} = f_D(A, H) \tag{1}$$

where H denotes the hidden embedding and $\tilde{G}$ denotes the reconstructed graph characteristics.

Given node initial embeddings x_i in the feature matrix X , we mask nodes as follows:

$$\tilde{x}_i = \begin{cases} x_{[M]} & v_i \in \tilde{V} \\ x_i & v_i \notin \tilde{V} \end{cases} \tag{2}$$

where $\tilde{V}$ are randomly-chosen masked nodes, emb_n is the embedding of node vi ready for training the graph representation module. This masking process is only applied during training. During detection, we do not mask any nodes. Then the corrupted graph is fed into the graph encoder to generate representation H.

4.2.2. Graph encoder

The initial node embeddings obtained from the graph construction phase are based on primitive features only. However, these primitive features often fail to capture the complex behavior of system entities. Contextual information about system entities, such as their neighboring nodes, multi-hop relationships, and interaction patterns with other entities, is crucial for high-quality node embedding generation.

To this end, we adopt and extend the Graph Masked Auto-Encoder GraphMAE2 [28] to generate more semantic node representations in a self-supervised manner. The graph masked auto-encoder consists of an encoder and a decoder: the encoder generates context-aware embeddings by propagating and aggregating node features over the graph; the decoder attempts to reconstruct the original features to provide a supervised signal for training. The decoder performs the following three tasks: 1) Generate multiple corrupted codes through randomly re-masking the code, which are all forced to reconstruct the input features after GNN decoding. 2) An MLP-based decoder predicts latent target representations, which are generated by the target generator and the unmasked graph. 3) Meanwhile, reconstruct the structure by predicting the edges between nodes.

Empirical results from GraphMAE and GraphMAE2 show that the Graph Attention Network (GAT) [30] achieves superior embedding and outperforms other methods on node classification tasks. Therefore, we utilize a multi-layer stacked GAT as the encoder to obtain a more expressive node representation. GAT improves the ability to model inter-node relationships by introducing an attention mechanism that assigns learnable weights to different neighboring nodes, enabling the model to pay more attention to important nodes when aggregating neighboring features. By stacking multiple layers of GAT, the encoder captures deep graph structure information and multi-hop interaction patterns. The message-passing process in each GAT layer is defined as follows:

$$msg(src, dst) = W_{msg}^{T}(h_{src} || emb_e) \tag{3}$$

GAT incorporates the initial edge embedding emb_e to encode interaction-specific information between source and target nodes, thus enhancing its ability to model relational dependencies. An attention mechanism is then applied to compute the attention coefficients between the message source and its destination:

$$\alpha(src, dst) = \mathrm{LeakyReLU}\left(W_{ad}^{T} h_{dst} + W_{am}\mathrm{msg}(src, dst)\right) \tag{4}$$

$$a(src, dst) = Softmax(\alpha(src, dst)) \tag{5}$$

For each target node, GAT updates its embedding by aggregating information from neighboring nodes weighted by these normalized attention coefficients.

$$AGG(h_{dst}, h_{\mathcal{N}}) = W_{self} h_{dst} + \sum_{i \in \mathcal{N}_n} a(i, dst) msg(i, dst) \tag{6}$$

$$h_n^l = AGG^l\left(h_n^{l-1}, h_{\mathcal{N}_n}^{l-1}\right) \tag{7}$$

where h_n^l is the hidden embedding of node n at lth layer of GAT, h_n^{l-1} is that of layer $l-1$ and N_n the one-hop neighborhood of n.

To capture multi-hop relationships among nodes in the graph, we stack a multi-layer graph attention network (GAT) to enhance the node representations by aggregating information from local neighborhoods layer by layer. Instead of relying solely on the final layer's output, we concatenate hidden representations from all GAT layers to form the final node embedding:

$$h_n = \tilde{x}_n \left\| h_n^1 \right\| \cdots \| h_n^l \tag{8}$$

where || denotes the splicing operation. This aggregation strategy enables the model to integrate both local and global contextual information, thereby producing more expressive and semantically rich node embeddings.

4.2.3. Graph decoder

Reconstruction based solely on masked features tends to capture only shallow associations, lacking the capacity to model deeper semantic structures and contextual dependencies. As a result, the learned representations may be vulnerable to mimicry attacks or out-of-distribution (OOD) inputs, reflecting limited robustness and poor generalization. To address these limitations and mitigate overfitting in feature reconstruction, we introduce two regularization strategies during decoding: multi-view random re-mask decoding and latent representation prediction. In addition, considering that provenance graph contains extensive entity interactions that reflects the semantic associations and behavioral structure of the system under normal operation, we further introduce a structure reconstruction task to enhance the model's ability to learn graph structures. The decoder consists of the following three core components:

Multi-view random re-mask decoding. To improve feature reconstruction, We introduce randomness in the decoding and require the decoder to restore the input x from different and partially observed embeddings. The decoder maps the latent code H to the input feature space to reconstruct $X$for optimization. Multi-view random re-mask decoding strategy randomly re-masks the encoded representations before feeding them into the decoder. We resample a subset of nodes $\overline{\mathcal{V}} \subset \mathcal{V}$ following a uniform distribution. Note that the set of nodes re-masked in the second stage, denoted as $\overline{\mathcal{V}}$, differs from the initially masked node set $\tilde{V}$ defined in Eq. (2). During re-masking, all nodes (including those previously masked or unmasked) are equally likely to be selected, ensuring unbiased regularization. Then the corrupted representation matrix $\bar{H}$ is derived from H by replacing the node representations h_i of selected nodes v_i with a shared mask token [DMASK].

$$\bar{h}_i = \begin{cases} h_{[M]} & v_i \in \overline{\mathcal{V}} \\ h_i & v_i \notin \overline{\mathcal{V}} \end{cases} \tag{9}$$

Then, the decoder reconstructs the original input features X from corrupted representations. This procedure is repeated K times, with each round randomly re-masking the encoded embeddings to generate K views, each containing distinct contextual information. The decoder is required to reconstruct the input features from each of these views.

Finally, we adopt the scaled cosine error [27] as the reconstruction loss function, and accumulate the reconstruction errors across all K rounds for optimization. This loss adaptively scales between easy and hard samples, facilitating more efficient learning. The scaling effect is controlled by the scaled coefficient γ.

$$\mathcal{L}_{input} = \frac{1}{|\widetilde{\mathcal{V}}|} \sum_{j=1}^{K} \sum_{v_i \in \widetilde{\mathcal{V}}} (1 - \frac{x_i^\top z_i^{(j)}}{\|x_i\| \cdot \|z_i^{(j)}\|})^\gamma \tag{10}$$

Latent representation prediction. To further reduce the model's direct reliance on input features, we require it to predict the potential representation of masked nodes in the embedding space. We perform the prediction in representation space beyond input feature space. A target generator is employed to produce latent prediction targets from the unmasked graph.

During training, the unmasked graph is first passed through a target generator to obtain the latent target representations. The target generator shares the same architecture as the encoder (i.e., a GAT network) and projector, but maintains an independent set of parameters. Then, the masked graph is encoded and projected to obtain the predicted representations.

$$\bar{Z} = g(H;\theta), \bar{X} = g'(f'_E(A, X;\xi);\xi) \tag{11}$$

We learn the parameters θ of the encoder and projector by minimizing the following scaled cosine error with gradient descent.

$$\mathcal{L}_{latent} = \frac{1}{N} \sum_{i}^{N} (1 - \frac{\bar{z}_i^\top \bar{x}_i}{\|\bar{z}\| \cdot \|\bar{x}\|})^\gamma \tag{12}$$

We define the overall feature reconstruction loss as the sum of the input loss and the latent loss:

$$\mathcal{L}_{fr} = \mathcal{L}_{input} + \mathcal{L}_{latent} \tag{13}$$

Sample-based structure reconstruction. Traditional graph self-encoders (e.g., GAE) often ignore the structural reconstruction task to reduce computational overhead. However, in system provenance graphs, the interaction behaviors among nodes play a key role in modeling the normal behavioral patterns of the system. Therefore, we employ sampling-based structural reconstruction to mine behavioral associations between nodes and enhance structural modeling capabilities by predicting edges between nodes.

Specifically, we randomly sample a certain number of real edges from the original graph as positive samples (positive edge pairs) and construct unconnected node pairs through negative sampling as negative samples (negative edge pairs). Subsequently, the embeddings of the source and target nodes are extracted from the node representations output by the encoder and are concatenated and fed into a two-layer multilayer perceptron (MLP) to predict the existence of edges between them. The module trained using the Binary Cross Entropy loss (BCE) for supervised optimization of the error between the prediction scores and ground-truth labels.

$$prob(src, dst) = \sigma(MLP(h_{src} || h_{dst})) \tag{14}$$

$$L_{sr} = -\frac{1}{|\hat{N}|} \sum_{n \in \hat{N}} (log(1 - prob(n, n^-)) + log(prob(n, n^+))) \tag{15}$$

where (n, n^-) and (n, n^+) are negative and positive samples.

Finally, the final objective function combines the feature reconstruction loss and the structure reconstruction loss:

$$\mathcal{L} = \mathcal{L}_{\text{fr}} + \mathcal{L}_{\text{sr}} \tag{16}$$

4.3. Threat detection

After obtaining the output embedding from the graph representation module, we employ a lightweight unsupervised anomaly detection module. Its core idea is to identify potentially malicious nodes or system states by modeling the degree of deviation of the sample distribution in the embedding space using the K-Nearest Neighbors (KNN) method.

The detection module does not require a complex learning process in the training phase, but only needs to memorize the embeddings of benign samples and organize them into a K-d tree index structure to accelerate subsequent queries. After training, the module performs the following three steps to realize outlier detection:

1) **K-nearest neighbor search:** For each embedding representation of a test sample, first retrieve its K-nearest neighbors from the K-D tree constructed over the training embedding.
2) **Similarity calculation:** Compute the average distance between the test embedding and its K-neighbors. To make this distance comparable across different regions of the embedding space, normalize it by the average neighbor distance observed for the corresponding neighbors in the training set. The resulting normalized value is used as the anomaly (outlier) score.
3) **Outlier filtering:** If this outlier score exceeds a predefined threshold θ, the corresponding system entity or state is deemed anomalous and flagged as a potential threat. This detection module can be adapted to different granularity of detection tasks: batch-level detection and entity-level detection.

4.4. Attack reconstruction

After embedding-based anomalous node detection, the threat detection model typically outputs a set of nodes identified as malicious. However, these malicious nodes may be dispersed throughout the system provenance graph, making it difficult to directly reconstruct the attack paths, thus increasing the burden on security analysts to perform manual investigation.

To this end, we design and introduce a TTP rule-based attack scenario reconstruction method, which aims at analyzing and correlating the detection results and automatically recovering more explanatory attack paths. Considering that the attack behaviors in the system provenance graph are mostly manifested as process-initiated operations, we take the process-like nodes detected as malicious as the starting point of reconstruction and execute the construction process of the attack chain, which is shown in Algorithm 1.

In the reconstruction phase of the attack scenario, it is necessary to combine the mapping dictionary of node IDs to semantic attributes (e.g., process names, file paths, etc.) established in the embedding phase to provide the basic support for subsequent contextual analysis. The method first uses the detected high outlier nodes as seed nodes and adopts a forward and backward K-hop neighbor expansion strategy to

Algorithm 1 Threat context-based attack scenario reconstruction.

Input: Provenance Graph $PG = (V, E)$, k_{hops}, Max_n, Max_e, detected nodes $M = \{n_1, n_2, \ldots\}$, TTP rule base $\mathcal{R}$
Output: Attack scenario Graph $S = \{SG_1, SG_2, \ldots\}$

Initialize $S \leftarrow \emptyset$
for each malicious node m in M **do**
  if $type(m) = process$ and $confidence(m) > \theta$ **then**
    $Neighbor(m) = \{v \mid v \in PG, d(m, v) \leq k_{hops}\}$
    $SG = (Neighbor(m), Conn_{edge})$ ▷ construct suspicious subgraph
    Suspicious Subgraph Reduction: Remove redundant edges in $Conn_{edge}$ of SG
    if $SG_{nodes} \leq Max_n$ **and** $SG_{edges} \leq Max_e$ **then**
      Attack Stage Annotation:
        $\forall e \in SG$, label$(e) = \{r \in \mathcal{R} \mid match(e, r) = 1\}$
        TTP$(e) = \{\tau(r) \mid r \in label(e)\}$
        Stage$(e) = \{\sigma(r) \mid r \in label(e)\}$
      Stage Correlation:
        $SG = \langle Stage_1 \rightarrow Stage_2 \rightarrow \cdots \rightarrow Stage_k \rangle$
      $S \leftarrow S \cup \{SG\}$
    end if
  end if
end for

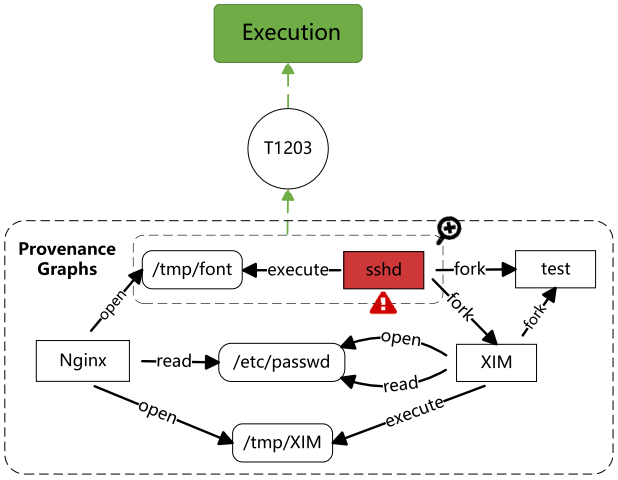


Fig. 4. An example of TTP matching from anomalous node behavior.

obtain its possible upstream and downstream interaction behaviors, and constructs the initial suspicious subgraph. To avoid "explosive growth" during neighborhood expansion (e.g., a two-hop neighborhood may contain tens of thousands of edges), we introduce edge reduction and filtering operations, so that only nodes and edges involved in malicious behavior are retained as much as possible. Redundant edges and unreachable nodes are pruned to further reduce the subgraph size. Additionally, lightweight intrusion detection heuristics are incorporated to support attack investigation, including whitelist IP filtering (e.g., trust_ip), detection of accesses to highly sensitive file paths (e.g., /etc/passwd), and matching Indicators of Compromise (IOCs) based on external threat intelligence. If the final subgraph does not exceed the set maximum node and edge limits, the subgraph is retained for subsequent analysis.

Security events are usually highly context-dependent and their interpretation often requires the incorporation of rich domain knowledge [31]. This method further incorporates the MITRE ATT&CK framework to match the behavioral patterns in suspicious subgraphs with TTP rules. This matching process draws on the strategy in the HOLMES [4] and Krystal [23] systems of automatically matching the corresponding attack techniques of nodes based on their contextual behavior towards each other.

Table 2
Dataset on streamspot.

Scenario	# of Graphs	Avg# of Nodes	Avg# of Edges
YouTube	100	8,292	113,229
Gmail	100	6,826	37,382
Video Game	100	8,636	112,958
Download	100	8,830	310,814
CNN	100	8,989	294,903
Attack	100	8,890	28,423

Table 3
Dataset on unicorn Wget.

Scenario	# Log pieces	Avg# of Nodes	Avg# of Edges
Benign	125	265,424	975,226
Attack	25	257,156	957,968

Table 4
Dataset on Darpa-E3.

Scenario	# Nodes	#Edges	Raw Data Size(GB)
E3-THEIA	1,623,966	2,874,821	17.9
E3-CADETS	1,627,035	3,303,264	18.3
E3-TRACE	3,288,676	4,080,457	15.4

As illustrated in Fig. 4, the detector flags an anomalous sshd process node and observes that it executes a suspicious file `/tmp/font`. This behavior can be automatically matched to T1203 (Execution tactic) in the ATT&CK matrix. This allows the abstraction of low-level system behaviors into higher-level attack semantics—attack stage. Eventually, the system will output a reconstructed attack scenario graph, which is labeled with attack techniques, attack stages, and contextual semantic information, thus reconstructing an attack scenario graph with a higher semantic level, more comprehensive information, and stronger interpretability. This not only helps in attack traceability and speculation of attack intent, but also assists in correcting false alarm nodes.

5. Evaluation

In this section, we conduct various experiments to validate the advantages of MGDA and answer the following research questions:

- **RQ1:** How does MGDA's detection performance compare to state-of-the-art methods?
- **RQ2:** How effective are the main components of MGDA?
- **RQ3:** What are the cost overheads of MGDA detection?
- **RQ4:** How does MGDA's attack scenario reconstruction perform?

5.1. Datasets settings

5.1.1. Datasets

StreamSpot dataset [14]: This dataset (See Table 2) comprises provenance graphs collected by SystemTap from six controlled environments, including CNN, Download, Gmail, Vgame, YouTube, and Attack, with each environment containing 100 provenance graphs. The Stream Spot dataset contains 600 batches of audit logs monitoring system calls under six unique scenarios. Five of the scenarios simulate benign user behavior and the attack scenario simulates a web Trojan.

Unicorn Wget dataset [10]: This dataset (See Table 3) includes simulated attacks designed by UNICORN, consisting of 150 batches of log data collected by CamFlow, of which 125 batches are benign and 25 are malicious.

DARPA-E3 dataset [15]: This dataset (See Table 4) is part of the DARPA TC program's third evaluation, containing several sub-datasets. We selected the CADETS, THEIA, and TRACE datasets to evaluate MGDA, using the same ground truth labels as Threatrace.

Table 5
Detection performance on different datasets.

Dataset	Ground Truth		#TN	#FN	#TP	#FP	Precision	Recall	FPR	F1-Score
	#Benign	#Malicious								
Unicorn Wget	25	25	24.01	1.75	23.25	0.99	96.11 %	93.00 %	3.96 %	94.46 %
StreamSpot	100	100	99.69	0.00	100	0.31	99.69 %	100 %	0.31 %	99.85 %
DARPA E3 CADETS	344,327	12,846	343,555	29	12,817	772	94.32 %	99.77 %	0.22 %	96.97 %
DARPA E3 THEIA	319,475	25,319	319,062	1	25,318	386	98.50 %	99.99 %	0.12 %	99.24 %
DARPA E3 Trace	616,021	68,086	615,512	14	68,072	509	99.26 %	99.98 %	0.08 %	99.62 %

5.1.2. Metrics

We use the evaluation metrics from Unicorn [10] and Threatrace [19] to compare performance across different granularities, enabling reasonable comparisons with other detection methods. The metrics include Precision (Pre), Recall (Rec), F1-score (F1), and False Positive Rate (FPR). The formulas for the metrics are as follows:

$$Precision = \frac{TP}{TP + FP} \tag{17}$$

$$Recall = \frac{TP}{TP + FN} \tag{18}$$

$$F1 - score = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{19}$$

$$FalsePositiveRate = \frac{FP}{FP + TN} \tag{20}$$

5.1.3. Implementation

To thoroughly evaluate MGDA, we address the following research questions. All experiments were performed on a machine equipped with 64 Intel vCPUs, 64 GB RAM, an NVIDIA A100 GPU, and Ubuntu 22.04.5 LTS.

For the hyperparameters of the model, the number of neighbors k is set at 10, the learning rate is set to 0.001, the weight decay factor is 5×10^4, the number of graph encoder layers and the number of multi-view re-randomization masks are both set to 3, and the mask rate is set to 0.5. The model is trained using the Adam optimizer. Regarding the embedding dimensions, we set the output embedding dimension d to 256 for batched log level detection and 64 for entity level detection, in order to balance model capacity and resource consumption.

5.2. Overall detection efficacy comparsion(RQ1)

Table 5 shows the anomaly detection performance of our proposed MGDA method on multiple datasets. The results demonstrate that MGDA effectively detects APT attacks with high accuracy under various scenarios. At the graph-level detection, MGDA achieves excellent precision scores of 99 % and 96 % on the StreamSpot and Unicorn Wget datasets, respectively, indicating its capability to accurately identify anomalous patterns in overall system behavior. . In the node-level anomaly detection task, MGDA also performs favorably on the CADETS, THEIA, and TRACE datasets, especially exhibiting lower False Negative (FN) rates and higher Recall. This highlights that MGDA is able to cover a wide range of attack types more comprehensively, thus effectively mitigating the security risk posed by undetected threats.

Further comparisons with state-of-the-art benchmarks are shown in Table 6. In the graph-level detection task, for the Unicorn Wget dataset, we compared with the Unicorn, Threatrace, and Magic methods; For the Streamspot dataset, we performed a full comparison with the original Streamspot method, Unicorn, Threatrace, and Magic; In the node-level detection task, for the DARPA E3 dataset, we evaluate against the Threatrace and Magic methods.

The results show that MGDA achieves the best or near-best detection performance across all datasets, fully verifying its generalization ability and robustness in different scenarios. Among them, in node-level detection, MGDA achieves a 4 % improvement in Precision compared to other methods on the CADETS dataset, and at the same time performs well in false positive rate (FPR), which indicates that the model maintains a higher detection coverage while effectively reducing the misclassification of normal nodes. In addition, our method outperforms state-of-the-art baselines in terms of recall, implying that the model is able to detect more real attack nodes with more comprehensive coverage. In real security scenarios, False Negative (FN) is often more threatening than False Positive (FP), which means that the real threats are not recognized in time, which may lead to serious consequences. The low FN and FPR characteristics of MGDA demonstrate its practical value in terms of precision and reliability, which can effectively assist the subsequent attack response and scenario traceability analysis. Overall, MGDA demonstrates a low miss rate and false alarm rate, reinforcing its applicability for real-world threat detection and investigation.

Table 6
Comparison with other APT detection methods on different datasets.

Dataset	Approach	Precision	F1	Recall	FPR
Unicorn Wget	Unicorn	86 %	90 %	95 %	15.5 %
	Threatrace	93 %	**95 %**	**98 %**	7.4 %
	MAGIC	95 %	94 %	93 %	5.48 %
	Ours	**96.11 %**	94.46 %	93 %	**3.96 %**
StreamSpot	StreamSpot	73 %	81 %	91 %	6.6 %
	Unicorn	95 %	96 %	93 %	1.6 %
	Threatrace	98 %	99 %	99 %	0.4 %
	MAGIC	99 %	99 %	99 %	0.82 %
	Ours	**99.69 %**	**99.85 %**	**100 %**	**0.31 %**
DARPA E3 CADETS	Threatrace	90 %	95 %	99 %	**0.2 %**
	MAGIC	81.53 %	89.73 %	99.77 %	0.84 %
	Ours	**94.32 %**	**96.97 %**	**99.77 %**	0.22 %
DARPA E3 THEIA	Threatrace	87 %	93 %	99 %	0.14 %
	MAGIC	98.48 %	99.23 %	99.99 %	0.12 %
	Ours	**98.50 %**	**99.24 %**	**99.99 %**	**0.12 %**
DARPA E3 TRACE	Threatrace	90 %	95 %	99 %	0.2 %
	MAGIC	96.87 %	98.40 %	99.98 %	0.36 %
	Ours	**99.26 %**	**99.62 %**	**99.98 %**	**0.08 %**

5.3. Ablation study (RQ2)

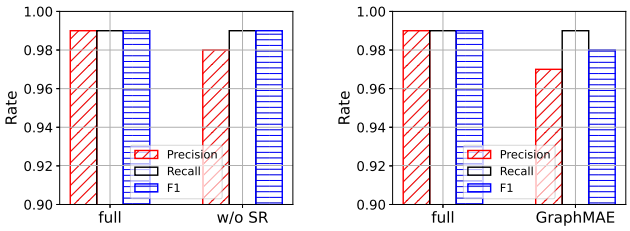
We conducted an ablation study on the TRACE dataset to evaluate the individual contributions of different submodules within the MGDA framework.

Impact of Structure Reconstruction(SR): We first removed the structure reconstruction module from MGDA. As shown in Fig. 5(a), this removal leads to a decrease in precision, indicating that structural information plays a role in enhancing node representation and improving anomaly detection performance.

Impact of multi-view random re-mask decoding: Subsequently, we replaced GraphMAE2 with the original GraphMAE. As shown in Fig. 5 (b), the precision of the GraphMAE-based model has decreased by 2 %. This demonstrates that the Multi-view random re-mask decoding strategy in GraphMAE2 effectively enhances the model's ability to generalize benign behaviors and helps better distinguish between normal and anomalous entities.

5.4. Performance overhead(RQ3)

Regarding computational efficiency, Table 7 reports that MGDA completes training on the E3-Trace sub-dataset within 863 s



(a) SR (b) GraphMAE

Fig. 5. Ablation study.

Table 7
Performance overhead.

Moduel	Time consumption(s)	Peak Memory consumption (MB)
Training	863	9,085
Threat Detection	624	1,820

(approximately 14 min). MGDA adopts a lightweight embedding strategy, using only node and edge types as features for embedding, and with a semantics-preserving compression mechanism, which reduces redundant computations and ensures efficient model training. In contrast, many prior works rely on resource-intensive architectures that increase training time and memory consumption. For instance, ShadeWatcher [32] leverages TransR and GNN to detect APTs based on recommendation. However, the large number of edges in provenance graphs results in high computational overhead during embedding and message passing. APT-MGL [33] embeds node representations by incorporating node type, action, and interaction frequency, and fuses these features through a multi-head self-attention mechanism. While this design improves feature expressiveness, the gain is limited and comes at the cost of significantly longer training time and higher memory usage. Consequently, on the same dataset, ShadeWatcher takes roughly 24 h, whereas APT-MGL requires over 45 min. MGDA achieves nearly 100× speedup over ShadeWatcher and more than 3× faster runtime than APT-MGL.

Moreover, the E3-Trace sub-dataset is collected on a two-week cycle, generating about 1.37 GB of audit logs daily. Under this workload, MGDA requires only about 2 min per day to complete APT detection using CPU resources alone. This high efficiency not only verifies the practicality of MGDA in real deployment environments, but also highlights its strong real-time detection capability under resource-constrained conditions.

5.5. Reconstruction performance (RQ4)

We empirically analyze the attack scenario reconstruction capability using the Cadets sub-dataset as a case study. Fig. 6 below shows the attack scenario reconstruction done with the anomalous nodes detected by the model. Based on the anomalous nodes detected by the system, including process and file alerts such as sshd, /tmp/font, XIM, test, etc. We perform forward-backward event correlation on the alerted nodes, combined with the cropping and event sifting of the intermediate nodes, to construct a compact and representative suspicious subgraph. Then we combine the TTP rule design ideas in Homles to automatically associate high-risk operations with MITRE ATT&CK techniques and tactics, realizing the precise annotation of attack techniques and stages. In the generated attack scenario reconstruction diagram, it can be seen that the attacker performs multiple attack operations such as file creation, file execution, data leakage, and so on. The reconstructed attack scenario reveals the kill chain stages of the attack, such as the four stages of reconnaissance, defense evasion, execution, and penetration. It is worth noting that through the correlation relationship of the nodes in the graph, analysts can further manually screen and correct false positives. For example, in addition to the attack behaviors mentioned in the official attack reports, we also accurately captured covert attack operations such as reading etc/passwd. Although ground truth does not mark this node as an attack node, our attack graph reveals its important role in the attack path, suggesting that the attack graph structure helps to discover more potential attack links. We manually compared and calibrated the system's reconstructed CADETS attack process with the dataset's publicly available official attack reports. The results show that the reconstructed attack graph not only highly covers the main attack nodes and events specified in the report, but also supplements the discovery of hidden behaviors such as etc/passwd reading. This improves the comprehensiveness and accuracy of attack traceability. The above results validate the utility of this method in attack scenario reduction, malicious node identification, and contextual semantic complementation to increase interpretability. Based on the automatic TTP annotation and attack stage mapping, the security analysis team is able to grasp the whole picture of the attack in the shortest possible time, precisely locate the attack entry and exit nodes, and take emergency response actions such as blocking the outreach malicious IPs in a timely manner, thus significantly reducing the consequences of the attack.

6. Discussion and limitations

False positivies. While MGDA maintains a low false positive rate in most settings, it may still produce elevated false alarms in certain scenarios. For example, when the logs collected in the training phase do not adequately cover the latest benign behavioral patterns in the system, or when the training data is of low quality, noisy and biased, the model may misclassify unknown normal behaviors as abnormal. This phenomenon can be attributed to concept drift, where the evolving nature of system behavior leads to a mismatch between learned and current data distributions. This is a common challenge in anomaly detection systems, especially in real-world environments where the system is constantly evolving and the behavioral patterns are continuously changing. To mitigate this, MGDA can incorporate an adaptive mechanism that feeds the false alarm samples confirmed by security experts to the model for training to adapt to the system changes, thus reducing the probability of false alarms. Additionally, a dynamic anomaly threshold adjustment strategy can further enhance robustness to environmental changes and sustain long-term operational stability.

Adversarial attacks. MGDA shows strong capability in detecting covert attacks and identifying escape strategies of attackers that mimic benign behaviors, but its robustness against attackers with stronger a priori knowledge still deserves further exploration. We assume that the attacker cannot directly access the structure and parameters of the MGDA model, and do not consider data poisoning attacks for the time being. However, the attacker may still launch evasion attacks by indirect means, e.g., tampering with audit logs, injecting forged nodes, or embedding sub-structures of legitimate process activities into the attacking subgraphs in order to disrupt the graph structure to hide malicious behaviors. It is found that MGDA shows strong resistance against attacks. It makes it difficult for adversarial attacks that tamper with features and malicious entity structures to be effective. Its robustness is mainly attributed to the following design aspects: first, the graph representation module of MGDA mainly relies on its neighbor information for the reconstruction of node features and adjacent structures in the functional feature and structure reconstruction phases. Second, the introduced multi-view mechanism enhances the model's ability to model semantic associations among nodes, thus improving the recognition of mimicry attacks. Lastly, self-supervised training based on the model makes the model more focused on the behavioral patterns, reducing the risk of being interfered or spoofed by specific labels. Therefore, for attackers to successfully launch adversarial attacks against MGDA, they must possess strong prior knowledge. Furthermore, they need to accurately manipulate the complex relationships between graph

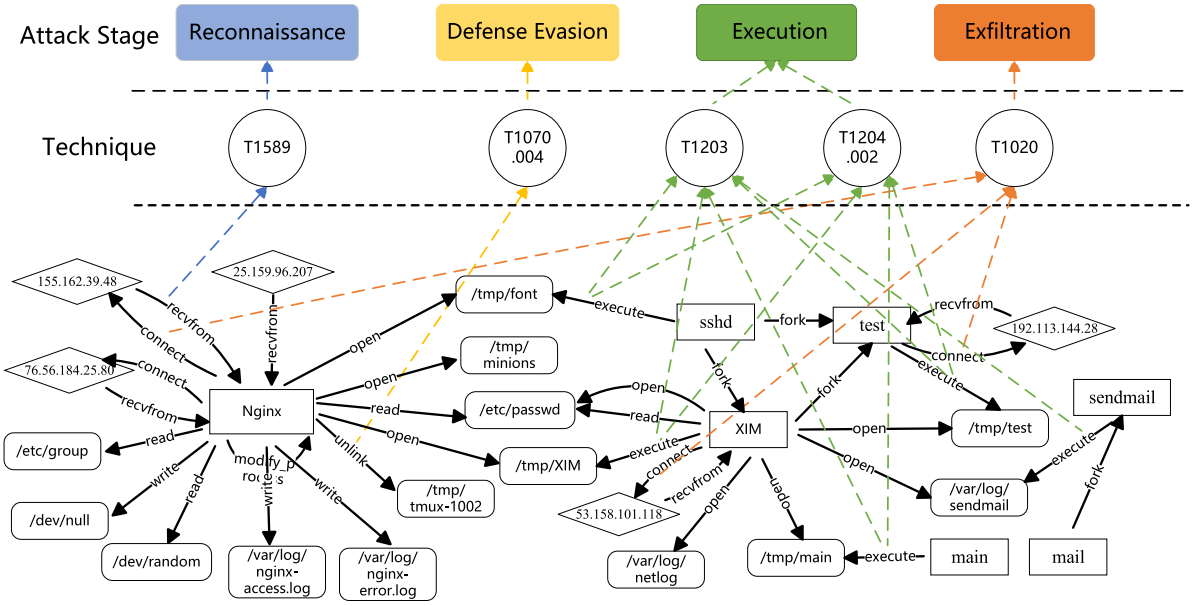


Fig. 6. Attack scenario from Nginx backdoor w/ Drakon in-memory.

structure, behavioral sequences, and embedding representations. These requirements greatly increase the attack cost and make practical implementation very difficult.

7. Conclusion

We propose MGDA, a self-supervised approach for efficient APT detection and interpretable attack scenario reconstruction. MGDA constructs provenance graphs by encoding nodes and edges with lightweight semantics. It models benign system behaviors through multi-view masked graph representation learning, enabling the reconstruction of both features and structures. An outlier detection method is then applied to efficiently identify anomalous nodes. Subsequently, MGDA performs automated correlation analysis and matches attack behaviors with tactics and techniques defined in the MITRE ATT&CK framework via designed TTP rules. This enables the reconstruction of attack scenario graphs enriched with high-level semantic representations, such as attack stages. We evaluated MGDA on three widely adopted datasets. Results show that MGDA achieves an average precision of 97.58 % and F1-score of 98.03 %, with up to 3× faster training speed compared to existing methods. MGDA not only maintains low computational overhead and exhibits strong detection capabilities, but also assists security analysts in gaining a comprehensive understanding of attack processes and conducting efficient traceability analysis.

CRediT authorship contribution statement

Mengjiao Cui: Writing – original draft, Software, Methodology, Investigation, Conceptualization; **Zhengwei Jiang:** Writing – review & editing, Resources, Formal analysis; **Shuai Li:** Methodology, Formal analysis; **Chunyan Ma:** Methodology, Formal analysis; **Kai Zhang:** Formal analysis; **Peian Yang:** Writing – review & editing, Resources, Methodology, Formal analysis; **Huamin Feng:** Writing – review & editing, Methodology, Formal analysis.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This research was supported by the Natural Science Foundation of China (No.62202466) and the Youth Innovation Promotion Association, CAS (No.2022159). This research was also partially supported by the Key Laboratory of Network Assessment Technology, the Chinese Academy of Sciences, and the Beijing Key Laboratory of Network Security and Protection Technology.

References

[1] A. Alshamrani, S. Myneni, A. Chowdhary, D. Huang, A survey on advanced persistent threats: techniques, solutions, challenges, and research opportunities, IEEE Commun. Surv. Tutorials 21 (2) (2019) 1851–1877.
[2] Z. Li, Q.A. Chen, R. Yang, Y. Chen, W. Ruan, Threat detection and investigation with system-level provenance graphs: a survey, Comput. Secur. 106 (2021) 102282.
[3] W.U. Hassan, A. Bates, D. Marino, Tactical provenance analysis for endpoint detection and response systems, in: 2020 IEEE Symposium on Security and Privacy (SP), 2020, pp. 1172–1189.
[4] S.M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, V.N. Venkatakrishnan, HOLMES: real-time APT detection through correlation of suspicious information flows, in: 2019 IEEE Symposium on Security and Privacy (SP), 2019, pp. 1137–1152.
[5] M.N. Hossain, S.M. Milajerdi, J. Wang, B. Eshete, R. Gjomemo, R. Sekar, S. Stoller, V.N. Venkatakrishnan, SLEUTH: real-time attack scenario reconstruction from COTS audit data, in: 26th USENIX Security Symposium (USENIX Security 17), USENIX Association, Vancouver, BC, 2017, pp. 487–504.
[6] S.M. Milajerdi, B. Eshete, R. Gjomemo, V.N. Venkatakrishnan, POIROT: aligning attack behavior with kernel audit records for cyber threat hunting, in: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS '19, Association for Computing Machinery, New York, NY, USA, 2019, p. 1795–1812.
[7] M.N. Hossain, S. Sheikhi, R. Sekar, Combating dependence explosion in forensic analysis using alternative tag propagation semantics, in: 2020 IEEE Symposium on Security and Privacy (SP), 2020, pp. 1139–1155.
[8] W.U. Hassan, S. Guo, D. Li, Z. Chen, K. Jee, Z. Li, A. Bates, Nodoze: combatting threat alert fatigue with automated provenance triage, in: 26th Annual Network and Distributed System Security Symposium, NDSS, 2019.
[9] Q. Wang, W.U. Hassan, D. Li, K. Jee, X. Yu, K. Zou, J. Rhee, Z. Chen, W. Cheng, C.A. Gunter, et al., You are what you do: hunting stealthy malware via data provenance analysis, in: NDSS, 2020.
[10] X. Han, T. Pasquier, A. Bates, J. Mickens, M.I. Seltzer, Unicorn: runtime provenance-based detector for advanced persistent threats, in: 27th Annual Network and Dis-

tributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23deldDel-deliIns–26, 2020, The Internet Society, 2020.
[11] A. Alsaheel, Y. Nan, S. Ma, L. Yu, G. Walkup, Z.B. Celik, X. Zhang, D. Xu, ATLAS: a sequence-based learning approach for attack investigation, in: 30th USENIX Security Symposium (USENIX Security 21), 2021, pp. 3005–3022.
[12] Z. Cheng, Q. Lv, J. Liang, Y. Wang, D. Sun, T. Pasquier, X. Han, Kairos: practical intrusion detection and investigation using whole-system provenance, in: 2024 IEEE Symposium on Security and Privacy (SP), 2024, pp. 3533–3551.
[13] M. Ur Rehman, H. Ahmadi, W. Ul Hassan, Flash: a comprehensive approach to intrusion detection via provenance graph representation learning, in: 2024 IEEE Symposium on Security and Privacy (SP), 2024, pp. 3552–3570.
[14] E. Manzoor, S.M. Milajerdi, L. Akoglu, Fast memory-efficient anomaly detection in streaming heterogeneous graphs, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '16, Association for Computing Machinery, New York, NY, USA, 2016, p. 1035–1044.
[15] DARPA, Transparent computing program engagement 3 data release, 2021, (https://github.com/darpa-i2o/Transparent-Computing). Accessed: 2025-07-02.
[16] MITRE, ATT&CK, 2025, (https://attack.mitre.org/).
[17] W.U. Hassan, D. Li, K. Jee, X. Yu, K. Zou, D. Wang, Z. Chen, Z. Li, J. Rhee, J. Gui, A. Bates, This is why we Can't cache nice things: lightning-fast threat hunting using suspicion-based hierarchical storage, in: Proceedings of the 36th Annual Computer Security Applications Conference, ACSAC '20, Association for Computing Machinery, New York, NY, USA, 2020, p. 165–178.
[18] Y. Liu, M. Zhang, D. Li, K. Jee, Z. Li, Z. Wu, J. Rhee, P. Mittal, Towards a timely causality analysis for enterprise security, in: NDSS, 2018.
[19] S. Wang, Z. Wang, T. Zhou, H. Sun, X. Yin, D. Han, H. Zhang, X. Shi, J. Yang, Threatrace: detecting and tracing host-based threats in node level through provenance graph learning, IEEE Trans. Inf. Forensics Secur. 17 (2022) 3972–3987.
[20] Z. Jia, Y. Xiong, Y. Nan, Y. Zhang, J. Zhao, M. Wen, MAGIC: Detecting advanced persistent threats via masked graph representation learning, in: 33rd USENIX Security Symposium (USENIX Security 24), USENIX Association, 2024, pp. 5197–5214.
[21] W. Cheng, T. Zhu, C. Xiong, H. Sun, Z. Wang, S. Jing, M. Lv, Y. Chen, SoK: knowledge is all you need: accelerating last mile delivery for automated provenance-based intrusion detection with LLMs, arXiv:2503.03108 (2025).
[22] T. Zhu, J. Yu, C. Xiong, W. Cheng, Q. Yuan, J. Ying, T. Chen, J. Zhang, M. Lv, Y. Chen, et al., Aptshield: a stable, efficient and real-time apt detection system for linux hosts, IEEE Trans. Dependable Secure Comput. 20 (6) (2023) 5247–5264.
[23] K. Kurniawan, A. Ekelhart, E. Kiesling, G. Quirchmayr, A.M. Tjoa, KRYSTAL: knowledge graph-based framework for tactical attack discovery in audit data, Comput. Secur. 121 (2022) 102828.
[24] T. Pasquier, X. Han, M. Goldstein, T. Moyer, D. Eyers, M. Seltzer, J. Bacon, Practical whole-system provenance capture, in: Proceedings of the 2017 Symposium on Cloud Computing, SoCC '17, Association for Computing Machinery, New York, NY, USA, 2017, p. 405–418.
[25] M.N. Hossain, J. Wang, R. Sekar, S.D. Stoller, {Dependence-preserving} data compaction for scalable forensic analysis, in: 27th USENIX Security Symposium (USENIX Security 18), 2018, pp. 1723–1740.
[26] T. Chen, C. Zheng, T. Zhu, C. Xiong, J. Ying, Q. Yuan, W. Cheng, M. Lv, System-level data management for endpoint advanced persistent threat detection: issues, challenges and trends, Comput. Secur. 135 (2023) 103485.
[27] Z. Hou, X. Liu, Y. Cen, Y. Dong, H. Yang, C. Wang, J. Tang, GraphMAE: self-supervised masked graph autoencoders, in: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD '22, Association for Computing Machinery, New York, NY, USA, 2022, p. 594–604.
[28] Z. Hou, Y. He, Y. Cen, X. Liu, Y. Dong, E. Kharlamov, J. Tang, GraphMAE2: a decoding-enhanced masked self-supervised graph learner, in: Proceedings of the ACM Web Conference 2023, WWW '23, Association for Computing Machinery, New York, NY, USA, 2023, p. 737–746.
[29] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, Bert: pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), 2019, pp. 4171–4186.
[30] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, Y. Bengio, Graph attention networks, arXiv:1710.10903 (2017).
[31] A. Ekelhart, E. Kiesling, K. Kurniawan, Taming the logs - Vocabularies for semantic security analysis, Procedia Comput. Sci. 137 (2018) 109–119. Proceedings of the 14th International Conference on Semantic Systems 10th – 13th of September 2018 Vienna, Austria.
[32] J. Zengy, X. Wang, J. Liu, Y. Chen, Z. Liang, T.-S. Chua, Z.L. Chua, SHADEWATCHER: recommendation-guided cyber threat analysis using system audit records, in: 2022 IEEE Symposium on Security and Privacy (SP), 2022, pp. 489–506.
[33] J. Ren, R. Geng, Provenance-based APT campaigns detection via masked graph representation learning, Comput. Secur. 148 (2025) 104159.

Mengjiao Cui recevied the BS degree from Beijing Information Science and Technology University in 2021. She is currently pursuing the phD degree with the Institute of Information Engineering, University of Chinese Academy of Sciences, Beijing, China. Her research interests include threat intelligence and threat detection.

Zhengwei Jiang received his PhD degree from the University of Chinese Academy of Sciences in 2014. He is currently a Senior Engineer at the Institute of Information Engineering, Chinese Academy of Sciences, and a professor at the School of Cyber Security, University of Chinese Academy of Sciences. His research interests include cyber threat intelligence and cyber threat detection.

Shuai Li received his BS degree from University of South China in 2022. He is presently pursuing the PhD degree at the School of Cyber Security, University of Chinese Academy of Sciences. His research interests include cyber threat intelligence and network security.

Chunyan Ma received her BS degree from Northwest Normal University in 2020. She is presently pursuing the PhD degree at the School of Cyber Security, University of Chinese Academy of Sciences. Her current research interests include cyber threat intelligence and cyber threat detection.

Kai Zhang received his PhD degree from the School of Information and Communication Engineering, Beijing University of Posts and Telecommunications in 2020. His research interests include network security and machine learning.

Peian Yang received his PhD degree in the University of Chinese Academy of Sciences in 2018. He is currently an Associate Senior Engineer at the Institute of Information Engineering, Chinese Academy of Sciences. His research interests include cyber threat intelligence and cyber threat attribution.

Huamin Feng received his PhD degree in 1984. He is currently a professor at the Beijing Electronic Science and Technology Institution and a visiting professor at the School of Cybersecurity, University of Chinese Academy of Sciences. His research interests include cyber security situational awareness and Internet Measurement.